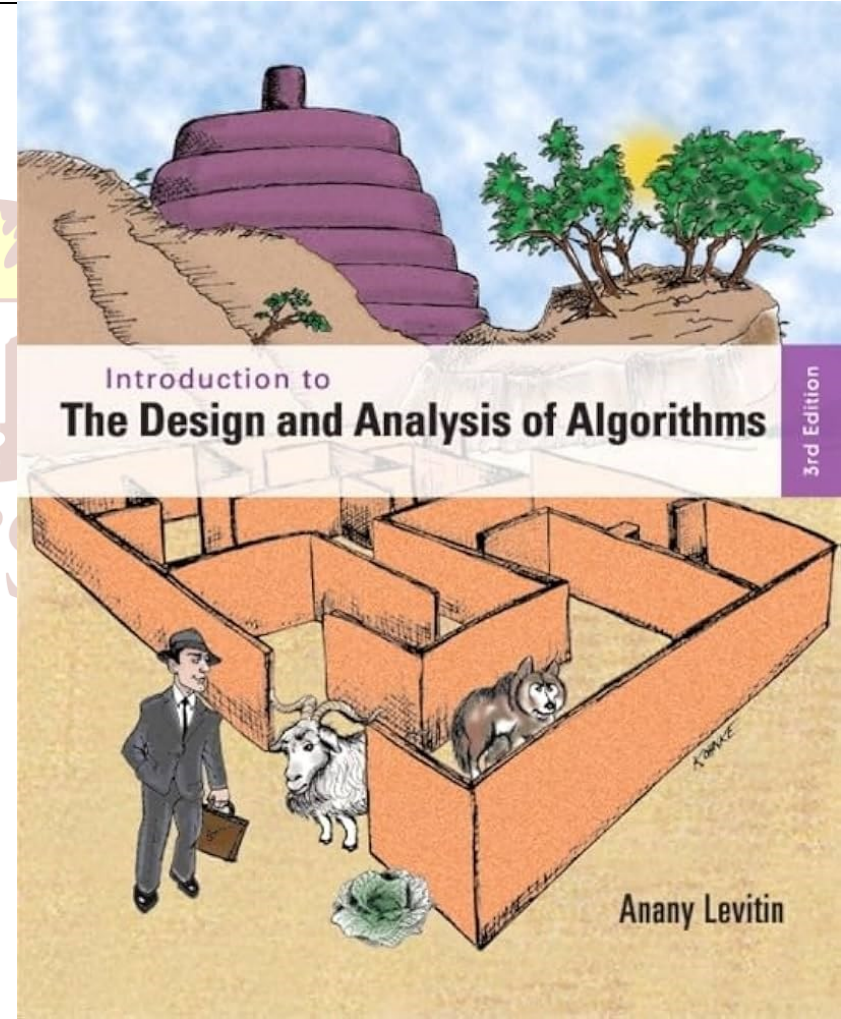


CS 07340 DESIGN AND ANALYSIS OF ALGORITHMS

WEEK 5 OVERVIEW

- Divide-and-Conquer
 - Master Theorem
 - MergeSort
 - QuickSort
 - BST Traversal

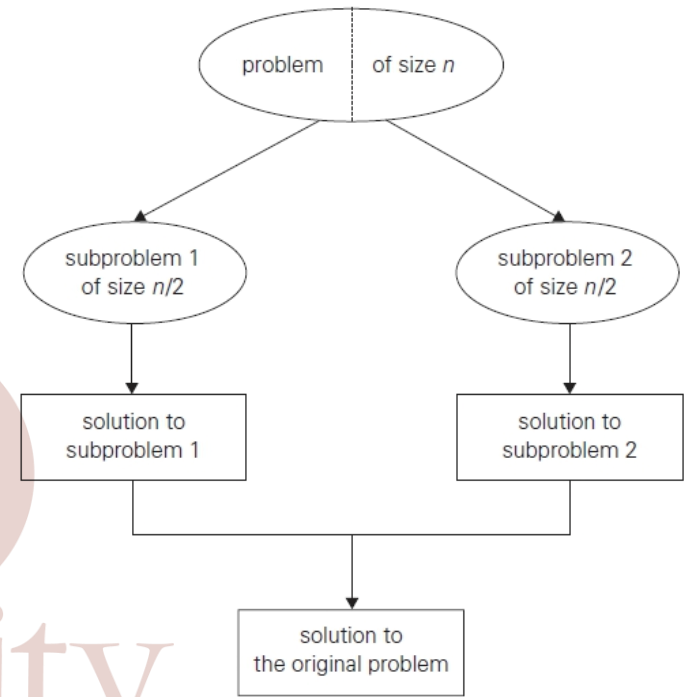
Rowan
University



DIVIDE-AND-CONQUER

Divide-and-conquer algorithms work according to the following general plan:

1. A problem is divided into several subproblems of the same type, ideally of about equal size.
2. The subproblems are solved (typically recursively, though sometimes a different algorithm is employed, especially when subproblems become small enough).
3. If necessary, the solutions to the subproblems are combined to get a solution to the original problem.



Example: Consider summing up eight numbers

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$$

The brute-force algorithm takes seven additions, which is $O(n)$ where n is the count of numbers. How about creating subproblems of size 2?

$$\begin{array}{ccccccc} & & 10 + 26 & & & & \\ & 3 + 7 & & 11 + 15 & & & \\ 1 + 2 & & 3 + 4 & & 5 + 6 & & 7 + 8 \end{array}$$

It still takes seven additions. The additions are actually the overhead here. There will always be an overhead $f(n)$ associated with subdividing a problem and then putting it back together.

MASTER THEOREM

We can apply the Master Method for recurrences of the form

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

where b is the number of subproblems and a of them need to be solved. It is a simplified form of generating functions theory, and the bounds are usually determined by the behavior of $f(n)$. This is a hard problem, in general.



Suppose we have the following recurrence for running time $T(n)$:

$$T(n) = a \cdot T\left(n/b\right) + f(n)$$

Master Theorem If $f(n) \in \Theta(n^d)$ where $d \geq 0$ in recurrence (5.1), then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Analogous results hold for the O and Ω notations, too.

ALTERNATE VERSION (CLRS)

Theorem 4.1 (Master theorem)

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

SKETCH OF PROOF FOR MASTER METHOD

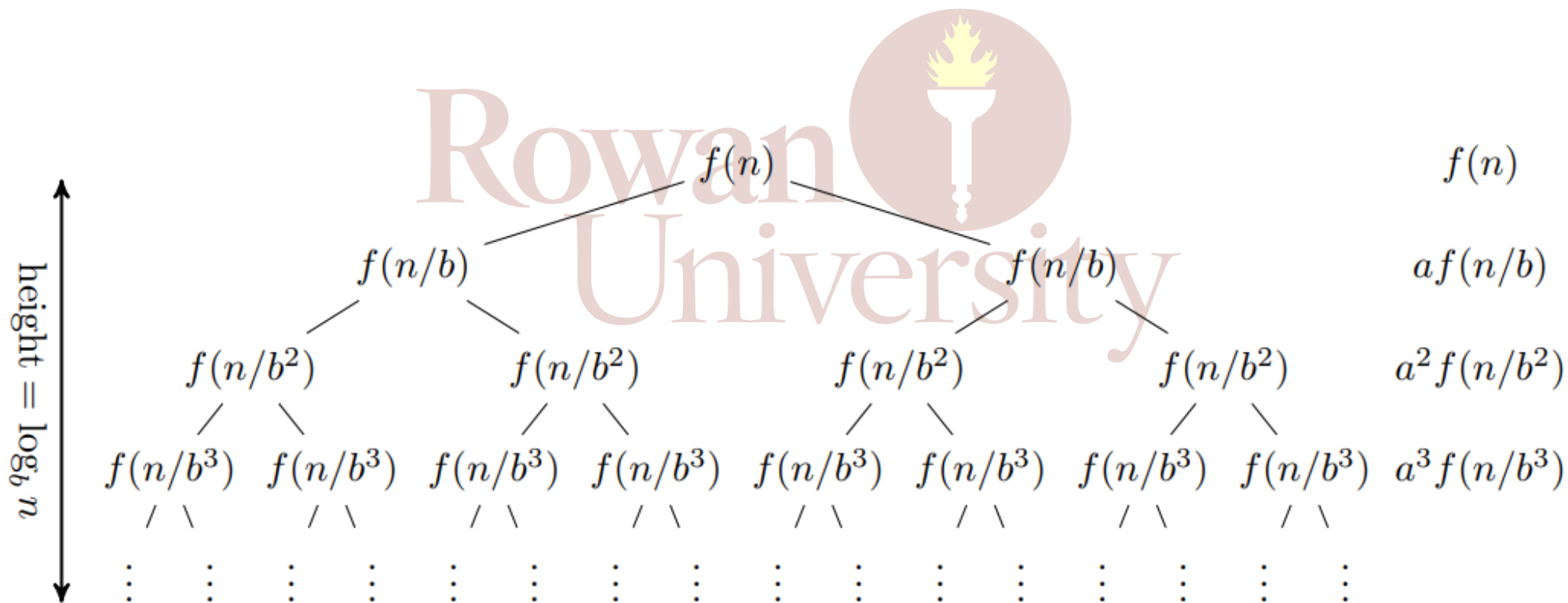
The recurrence

$$T(n) = a \cdot T(n/b) + f(n)$$

describes a divide-and-conquer approach which we can visualize with a tree where:

- Each node represents the time $f(n)$ needed as overhead when creating subproblems.
- Each layer represents the subproblems for the next iteration.
- The height of the tree represents the maximum number of subdivisions needed to reach a base case.
- We see that each layer has size a (in the graph $a = 2$).
- The height of the tree is $\log_b n$.

$$T(n) = \sum_{i=0}^{\log_b n} a^i f\left(n/b^i\right) + O\left(n^{\log_b a}\right)$$



Example: Use the Master Theorem on our earlier addition problem to show that the problem is $\Theta(n)$.



Solution: Let the input size $n = 2^k$. The overhead function $f(n) = 1$ (one operation of adding). Then

$$T(n) = 2T(n/2) + 1$$

with $a = 2$, $b = 2$, and $d = 0$. Hence $a > b^d$ and the runtime is $\Theta(n^{\log_b(a)}) = \Theta(n)$.



MERGESORT

MergeSort is another comparison-based sorting algorithm. It works as follows:

1. Divide the unsorted list recursively into halves, eventually reaching the base case of n sublists, each containing one element.
2. Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining. This will be the sorted list.

MergeSort does not work in-place since it requires $O(n)$ temporary memory. In-place MergeSort algorithms have been created but the trade-off is a slower runtime.

ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n - 1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A) //see below

ALGORITHM $Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])$

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted

//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C

$i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$

while $i < p$ **and** $j < q$ **do**

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]; i \leftarrow i + 1$

else $A[k] \leftarrow C[j]; j \leftarrow j + 1$

$k \leftarrow k + 1$

if $i = p$

 copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

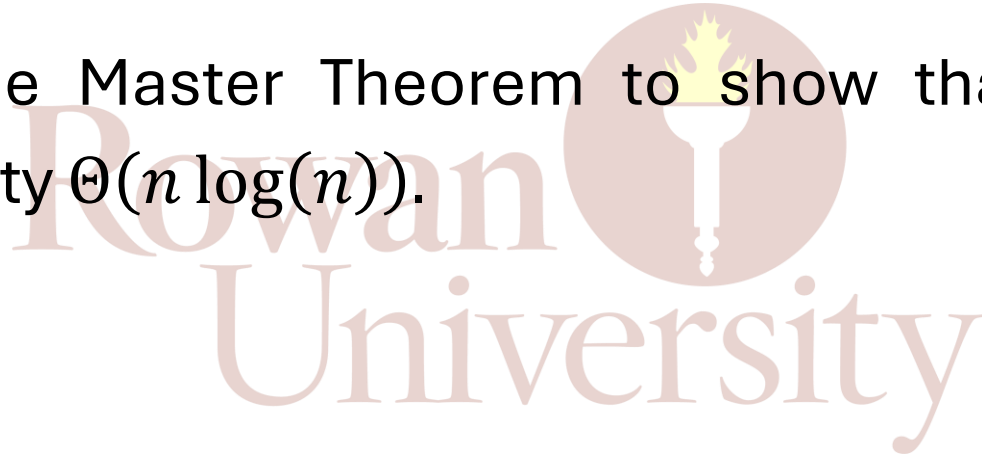


Example: Program standard *MergeSort* in Java.



The normal implementation of *MergeSort* has runtime $\Theta(n \log(n))$.

Example: Use the Master Theorem to show that MergeSort has runtime complexity $\Theta(n \log(n))$.



QUICKSORT

QuickSort is another divide-and-conquer comparison sort algorithm. It works by selecting a pivot element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. It has worst case $O(n^2)$ comparisons when using a poor choice of pivot, but on average its expected runtime is $O(n \log(n))$.

ALGORITHM *Quicksort*($A[l..r]$)

//Sorts a subarray by quicksort

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right

// indices l and r

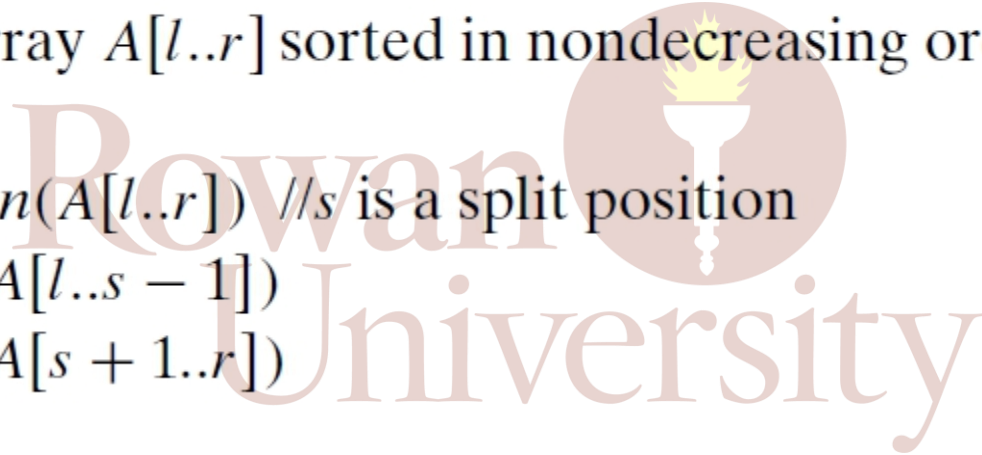
//Output: Subarray $A[l..r]$ sorted in nondecreasing order

if $l < r$

$s \leftarrow \text{Partition}(A[l..r])$ // s is a split position

Quicksort($A[l..s - 1]$)

Quicksort($A[s + 1..r]$)



The split position s is usually called a ***pivot***.

For *QuickSort*, we **rearrange** the array such that, when the two subarrays are sorted, the whole array is ordered. The position of the partition depends on the contents of the array. We do the two recursive calls **after** working on the whole array.

- Divide

- Partition and rearrange $A[s \dots r]$ into two (possibly empty) subarrays $A[s \dots q - 1]$ and $A[q + 1 \dots r]$ such that each element in $A[s \dots q - 1]$ is less than or equal to each element in $A[q + 1 \dots r]$

- Conquer

- Sort both subarrays $A[s \dots q - 1]$ and $A[q + 1 \dots r]$ recursively

- Combine

- Concatenate the two arrays

PARTITION ALGORITHM FROM CLRS

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```



The key to the algorithm is the Partition method. In this implementation, we select $x = A[r]$ (the last element) as a pivot. The probability of x being a “good” pivot (median value) is the same as being “bad” (smallest or largest) since we assume a uniform distribution.

- $x = A[r]$ is the pivot.
- The loop swaps elements that are smaller than the pivot into positions in front (possibly onto themselves).
- We end up with a partition {unordered < pivot}{pivot}{unordered ≥ pivot}.

Example: Sort {16, 4, 10, 14, 7, 9, 3, 2, 8, 1} using *QuickSort*.



- Round 1:

- Pivot $q = 9$, $A[9] = 1$
- Partition $\{ \ } \{1\} \{4,10,14,7,9,3,2,8,16\}$ with index 0 since our pivot was the minimum

- Round 2:

- Pivot $q = 9$, $A[9] = 16$ (maximum)
- Partition $\{ \ } \{1\} \{4,10,14,7,9,3,2,8\} \{16\} \{ \ }$ with index 9 since our pivot was the maximum

- Round 3:

- Pivot $q = 8$, $A[8] = 8$ (maximum)
- Partition $\{ \ } \{1\} \{4,7,3,2\} \{8\} \{14,10,9\} \{16\} \{ \ }$ with index 5

- Round 4.1.1:
 - Pivot $q = 4$, $A[4] = 2$ (minimum)
 - Partition { } {1} { } {2} {7,3,4} {8} {14,10,9} {16} { } with index 1
- Round 4.1.2:
 - Pivot $q = 4$, $A[4] = 4$ (maximum)
 - Partition { } {1} {2} {3} {4} {7} {8} {14,10,9} {16} { }
- Round 4.2.1:
 - Pivot $q = 8$, $A[8] = 9$ (minimum)
 - Partition { } {1} {2} {3} {4} {7} {8} { } {9} {10,14} {16} { }
- Round 5:
 - Pivot $q = 8$, $A[8] = 14$ (maximum)
 - Partition { } {1} {2} {3} {4} {7} {8} { } {9} {10} {14} { } {16} { }

```

import java.util.*;

public class QuickSort<T extends Comparable<T>>
{
    // Java cannot handle generic types in
    // arrays due to type erasure. Proper arrays
    // have type information stored at runtime.
    // Need to use a different container that is
    // type safe.
    private List<T> A;
    private final Comparator<T> comparator;

    public QuickSort(List<T> unorderedArray) {
        A = new ArrayList<>(unorderedArray);
        comparator = new Comparator<T>() {
            @Override
            public int compare(T o1, T o2) {
                return o1.compareTo(o2);
            }
        };
    }

    public List<T> Sort() {
        Sort(0, A.size()-1);
        return (List<T>) new ArrayList(A);
    }
}

```

```

private void Sort(int p, int r) {
    if(p < r) {
        int q = Partition(p, r);
        Sort(p, q-1);
        Sort(q+1, r);
    }
    return;
}

private int Partition(int p, int r) {
    T x = A.get(r);
    int i = p-1;
    for(int j = p; j < r; j++) {
        int compareResult = comparator.compare(A.get(j), x);
        if(compareResult < 0) {
            i++;
            Collections.swap(A, i, j);
        }
    }
    return i+1;
}
}

```

RUNTIME ANALYSIS FOR QUICKSORT

QuickSort runs optimal when the divide step is balanced. In that case

$$T(n) = 2 \cdot T(\lfloor n/2 \rfloor) + \Theta(n)$$

The pivot is the median and we recursively call on two subarrays of size $\frac{n-1}{2}$. The [Master Theorem](#) gives us a bound of $O(n^{\log_2 2} \log(n)) = O(n \log(n))$.

WORST CASE SCENARIO

Our example ran into the worst case a few times: If the pivot is the minimum or maximum of the current subarray, then we get a linked list recurrence rather than a tree:

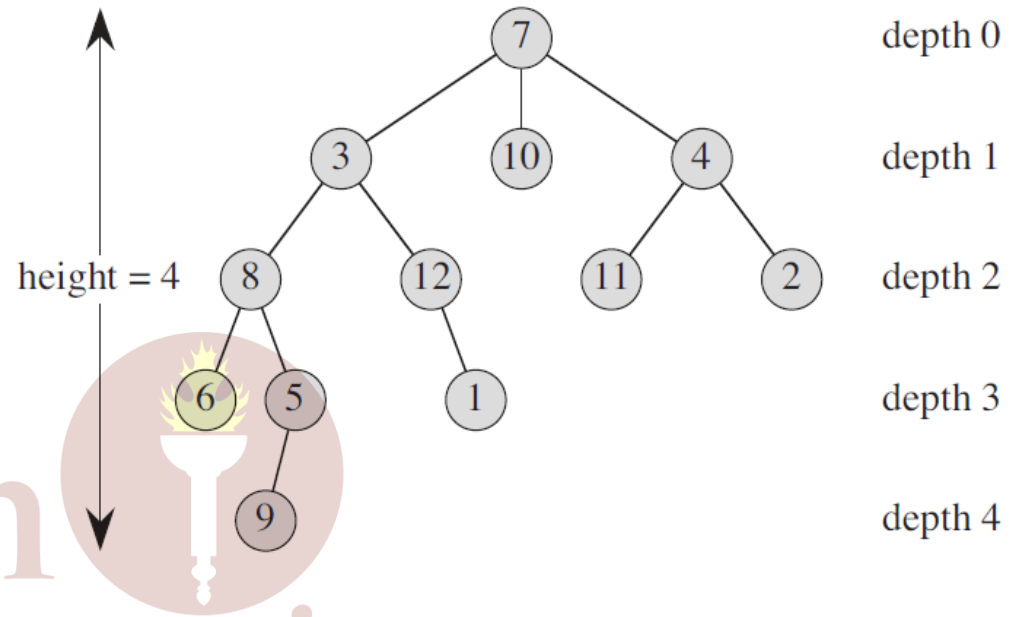
$$T(n) = T(n - 1) + T(0) + \Theta(n)$$

Here $\Theta(n)$ is the partitioning cost since we have to go through the array to see that there is nothing to be done. The recognition of the empty array takes $T(0) = \Theta(1)$ and can be folded into $\Theta(n)$. So we basically get

$$\begin{aligned} T(n) &= \sum_{i=1}^n (T(0) + \Theta(i)) \\ &= \Theta(n^2) \end{aligned}$$

ADT TREE

A tree is a connected undirected graph without cycles. Rooted trees have a special node called **root**. Trees can be represented graphically with nodes and edges such that nodes connected to the root (its **children**) are in a layer below the root. This can be extended to all nodes until we get a graph in which the well-defined distance of a node from the root determines its layer (called **depth**). A node connected to a node in a lower layer is called its **parent**. Nodes without children are called **leaves**. Only root has no parent.



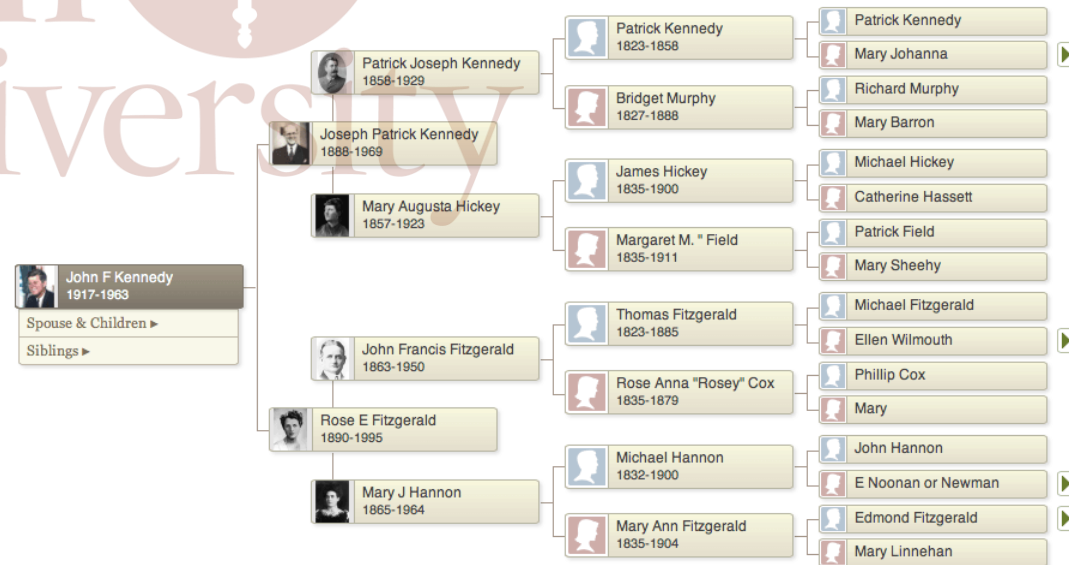
The length of the longest path from root to a leaf is called ***height*** of the tree. The structure below (and including) a node is called the ***subtree*** rooted at that node.

Trees are a widely used abstract data type in computer science. There are many specialized trees. Actual implementations are data structures. They require

- Memory locations/variables for each node
- References/pointers between nodes and children
 - Leaves *may* need to be identified
- Root needs to be identified

Family trees (upside down, with one descendent functioning as root and ancestors functioning as the children/descendants) are complete binary trees. Data **values** are the ancestors' information (as an object), and position (**key**) is determined by relation (姥姥 »lǎolao«: maternal grandmother, 姥爷 »lǎoye«: paternal grandmother).

Search in such a tree (called binary search trees due to their nature) is done by key, while the data is stored in the value.



SUMMARY OF ADT TREE DEFINITIONS

node element of the tree

root top node

child node connected and directly below given node

parent node connected and directly above child node

sibling nodes which share the same parent

ancestor parent, or parent's parent, or...

leaf node without children

subtree substructure below and connected to a given node

height length of longest path from root to any node

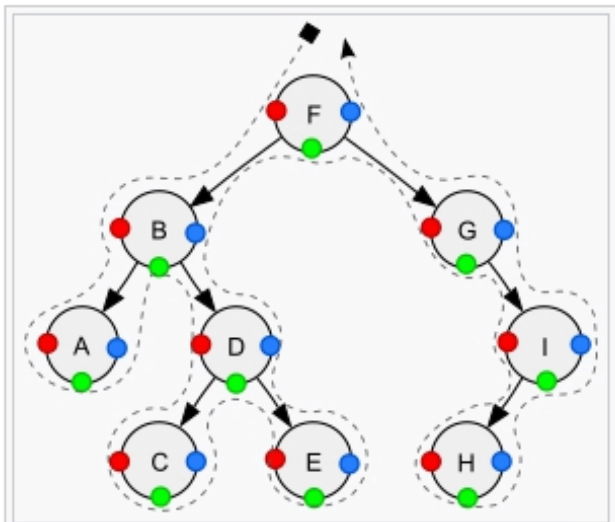
The minimal interface for ADT tree is usually

- Insert
- Delete
- Search

We can also include

- In-order walk
- Pre-order walk
- Post-order walk

to enumerate all items in the tree (with a tree traversal using Depth-First Search).



Depth-first traversal (dotted path) of a binary tree:

Pre-order (node visited at position red ●):

F, B, A, D, C, E, G, I, H;

In-order (node visited at position green ●):

A, B, C, D, E, F, G, H, I;

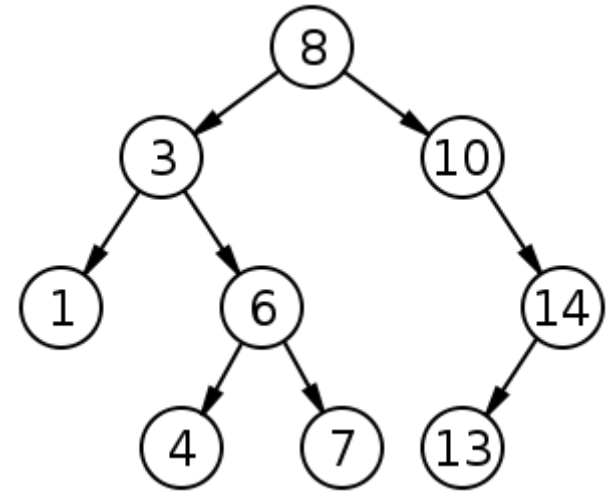
Post-order (node visited at position blue ●):

A, C, E, D, B, H, I, G, F.

BINARY SEARCH TREES

A **binary search tree** (BST, also called an ordered or sorted binary tree) is a linked-node based binary tree which stores key-value pairs (data) in each node. In addition, each node contains references to a

- left child (node)
- right child (node)
- parent (node) (optional, makes it doubly-linked)

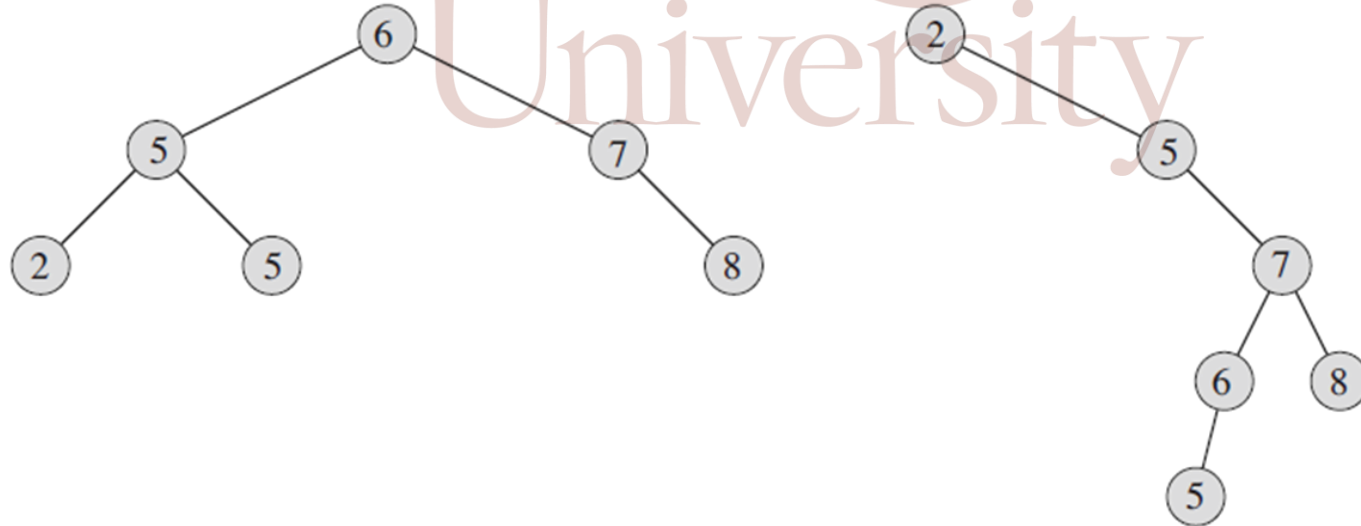


Nodes that are missing are referenced by **null** (Java) or **None** (Python). These are keywords to show that the object is not referring/pointing to anything. Left and right children are roots of left and right subtrees, respectively. Again, this approach is recursive in nature. All keys satisfy the following property:

- The left subtree contains **only** nodes with keys **less** than the node's key.
 - **node.left.key < node.key** (where **node** is the parent)
- The right subtree contains **only** nodes with keys **greater** than the node's key.
 - **node.key < node.right.key** (where **node** is the parent)
- The left and right subtree each must also be a binary search tree.

These two trees are both valid BSTs containing the same data (meaning keys – values are not listed in this example.) BSTs can take on the form of a tree where each node has only one child (a linked list).

Question: In which order should known/sorted data be inserted to achieve balance? We will look at self-balancing trees later.



Example (Exercise 5.3 #2): The following algorithm seeks to compute the number of leaves in a binary tree. Is this algorithm correct? If it is, prove it; if it is not, make an appropriate correction.

ALGORITHM *LeafCounter*(T)

//Computes recursively the number of leaves in a binary tree

//Input: A binary tree T

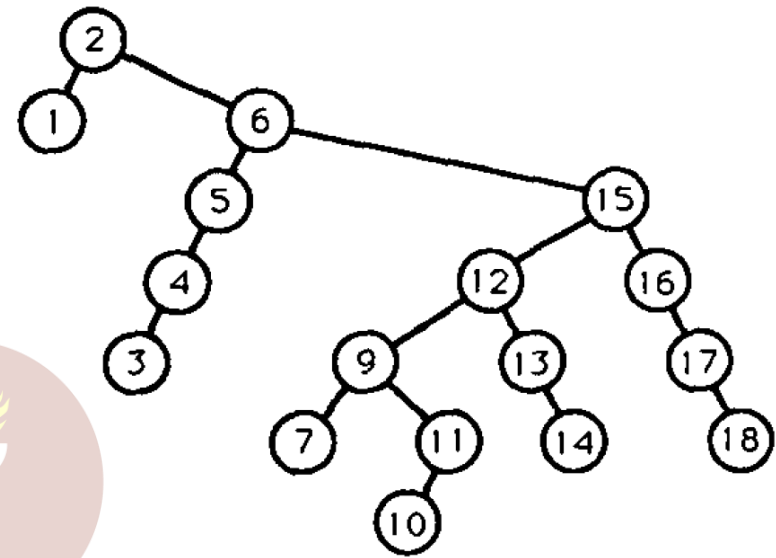
//Output: The number of leaves in T

if $T = \emptyset$ **return** 0

else return *LeafCounter*(T_{left}) + *LeafCounter*(T_{right})

BINARY SEARCH TREE TRAVERSAL

Drawing trees automatically is extremely challenging. However, once in a while it would be nice to have a method that prints the keys of a BST in a human-readable text form.



For example, the given tree from the Galperin/Rivest 1993 paper should yield the string

```
2(1,6(5(4(3, ), ),15(12(9(7,11(10, )),13( ,14)),16( ,17( ,18))))).
```

Example: Devise a **toString** method for a BST that prints keys as given.

Solution:

```
private void toStringRecursive(Node<Key, Value> node, StringBuilder rep) {
    if (node != null) {
        rep.append(node.key);
        if (node.left != null || node.right != null) {
            rep.append("(");
            toStringRecursive(node.left, rep);
            rep.append(",");
            toStringRecursive(node.right, rep);
            rep.append(")");
        }
    }
    else
        rep.append(" ");
}

// Java String is immutable so we need StringBuilder
@Override
public String toString() {
    StringBuilder representation = new StringBuilder("ScapegoatTree of size "+this.size()+" and height "+this.depth()+" : ");
    toStringRecursive(root, representation);
    return representation.toString();
}
```

MULTIPLICATION OF LARGE INTEGERS

The standard approach for multiplying two n -digit numbers has $O(n^2)$ multiplications (and $O(n)$ additions). A 1960 algorithm by then 23-year old student [Anatoly Karatsuba](#) provides a $O(n^{\log_2(3)})$ bound using divide-and-conquer techniques:

$$T(n) = 3 T(n/2) + (c_1 n + c_0)$$

for some constants c_1 and c_0 .

QUIZ TOPICS

- Runtime complexity of some standard sorting algorithms using the Master Theorem

